

Comparable vs Comparator

Java Interview Guide for SDET / Automation Engineers

Quick memory rule: Comparable = default/natural sorting defined inside the class. Comparator = custom or multiple sorting rules defined separately.

Comparable	Comparator
java.lang.Comparable	java.util.Comparator
Method: compareTo()	Method: compare()
Class implements Comparable<T>	Comparator is implemented separately
Defines natural/default ordering	Defines custom ordering
Usually one natural ordering	Can provide multiple sorting strategies
Example: Employee sorted by ID	Example: Employee sorted by name/salary

1. Comparable

Comparable is used when a class has a natural or default ordering. The class implements **Comparable<T>** and overrides **compareTo()**.

```
import java.util.*;

class Employee implements Comparable<Employee> {

    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public int compareTo(Employee other) {
        return Integer.compare(this.id, other.id);
    }

    @Override
    public String toString() {
        return id + " - " + name;
    }
}

public class ComparableExample {

    public static void main(String[] args) {

        List<Employee> employees = new ArrayList<>();

        employees.add(new Employee(103, "Rahul"));
        employees.add(new Employee(101, "Swapnil"));
        employees.add(new Employee(102, "Amit"));
```

```

        Collections.sort(employees);

        System.out.println(employees);
    }
}

```

Output: [101 - Swapnil, 102 - Amit, 103 - Rahul]

Key point: the Employee class itself defines its natural ordering.

2. Comparator

Comparator is used when you want a sorting rule that is different from the natural ordering, or when you need multiple sorting strategies. The comparison logic is kept outside the model class.

```

import java.util.Comparator;

class NameComparator implements Comparator<Employee> {

    @Override
    public int compare(Employee e1, Employee e2) {
        return e1.name.compareTo(e2.name);
    }
}

// Usage:
Collections.sort(employees, new NameComparator());

```

Result: employees are sorted alphabetically by name.

3. Comparator with Lambda

```

employees.sort(
    (e1, e2) -> e1.name.compareTo(e2.name)
);

```

Modern Java also provides `Comparator.comparing()`.

```

employees.sort(
    Comparator.comparing(e -> e.name)
);

```

4. Multiple Sorting Strategies

```

// Sort by name
employees.sort(Comparator.comparing(Employee::getName));

// Sort by ID
employees.sort(Comparator.comparingInt(Employee::getId));

// Sort by salary
employees.sort(Comparator.comparingDouble(Employee::getSalary));

// Sort salary in descending order
employees.sort(
    Comparator.comparingDouble(Employee::getSalary)
                .reversed()
);

// Sort by name, then ID
employees.sort(
    Comparator.comparing(Employee::getName)
                .thenComparingInt(Employee::getId)
);

```

5. compareTo() and compare() Return Values

Both methods follow the same basic comparison contract:

Return value	Meaning
Negative	First object comes before the second
0	Objects are equal for sorting
Positive	First object comes after the second

```
Integer.compare(10, 20); // negative
Integer.compare(20, 20); // 0
Integer.compare(30, 20); // positive
```

6. Important Coding Practice

Prefer Integer.compare() instead of subtraction when comparing integers.

```
// Avoid
return this.id - other.id;

// Prefer
return Integer.compare(this.id, other.id);
```

Using subtraction can produce incorrect results because of integer overflow for extreme values.

7. Practical SDET Example

Imagine an Employee object with these fields: ID, name and salary.

```
Comparable
↓
Default sorting → Employee ID

Comparator
  ■■■ Sort by name
  ■■■ Sort by salary
  ■■■ Sort by salary descending
  ■■■ Sort by name, then ID
```

8. Interview Question: Why Comparator?

If an Employee class already has ID as its natural ordering, but a test needs to sort employees by name, salary or another field, Comparator lets you add those sorting strategies without changing the natural ordering of Employee.

9. Interview Question: Comparable vs Comparator

Answer: Comparable is used to define the natural or default ordering of objects using **compareTo()**. The model class implements Comparable. Comparator is used to define custom or multiple sorting strategies using **compare()**, usually outside the model class. For example, Employee can use Comparable to sort by ID and separate Comparators to sort by name or salary.

10. Common Interview Follow-ups

- What is the difference between Comparable and Comparator?
- Why does Comparable use compareTo() while Comparator uses compare()?

- Can a class implement Comparable and also use Comparator?
- Can we create multiple Comparators for the same class?
- What does compareTo() return?
- What happens when compareTo() returns 0?
- How do you sort in descending order using Comparator?
- How do you sort using multiple fields?
- What is Comparator.comparing()?
- What is thenComparing()?
- Why should you avoid return this.id - other.id?
- How does Collections.sort() use Comparable?
- How does Collections.sort(list, comparator) work?
- What is natural ordering?
- How are Comparable and Comparator related to TreeSet and TreeMap?

11. Quick Revision

Comparable

Interface: java.lang.Comparable

Method: compareTo()

Purpose: Natural/default ordering

Logic: Usually inside the class

Example: Employee implements Comparable<Employee>

Comparator

Interface: java.util.Comparator

Method: compare()

Purpose: Custom/multiple ordering

Logic: Separate from the model class

Example: sort Employee by name, salary, etc.

■ Memory Trick

Comparable → Class compares itself.

Comparator → Separate comparator compares two objects.